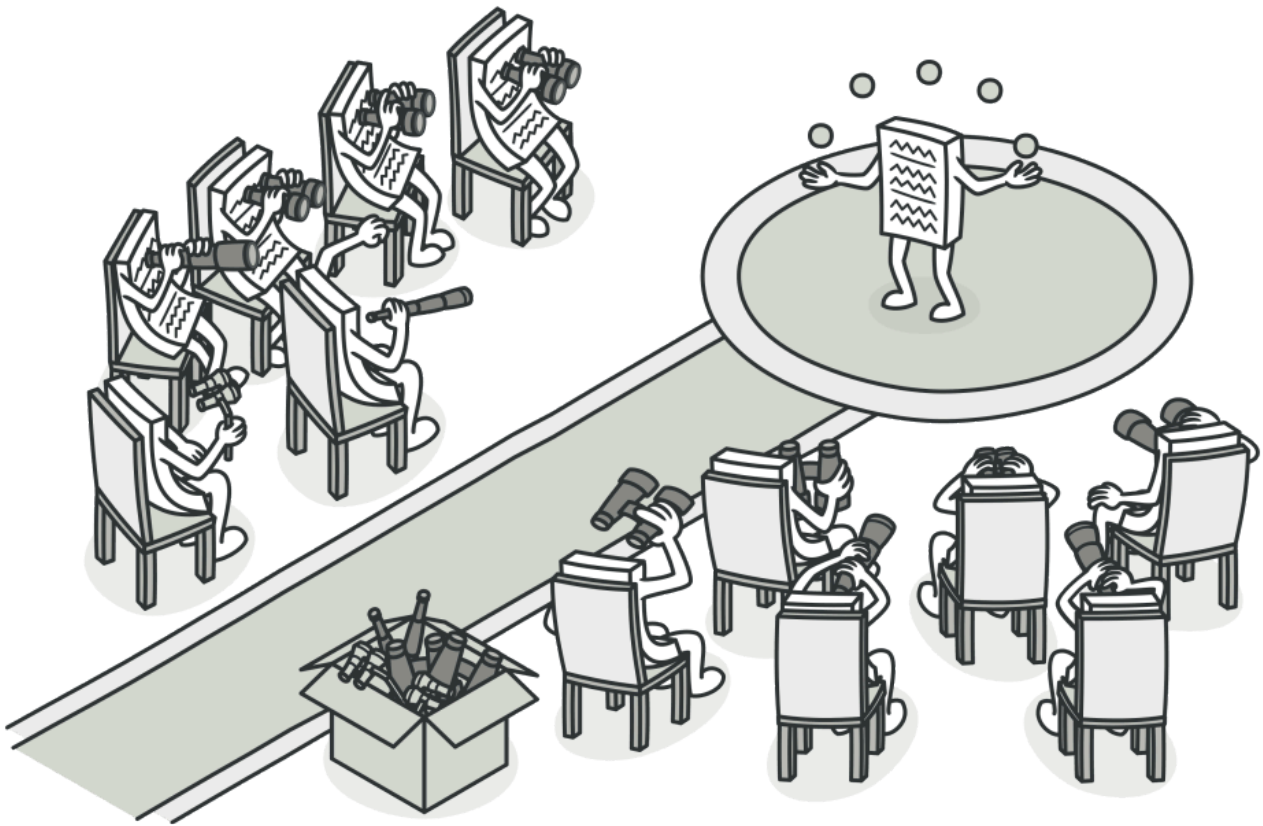


Observer



Notas organizadas a partir dos slides e das anotações de aula, Setembro/2026.

Disciplina: Métodos Avançados de Programação (MAP), UEPB — UEPB, 26.2

Professora: Luciana de Queiroz Leal Gomes

Aula 04: Padrões de Projeto GoF, Observer

Referências: GAMMA, HELM, JOHNSON e VLISSIDES, *Padrões de Projeto* ·
FREEMAN e FREEMAN, *Use a Cabeça! Padrões de Projetos*

Material Auxiliar

Padrões de Projeto

GoF Observer

- [Observer no Refactoring Guru](#) (site indicado em aula para estudar os padrões)

1. Retomando a aula anterior

A Aula de Strategy fechou com uma comparação entre herança e composição, retomada aqui como ponto de partida:

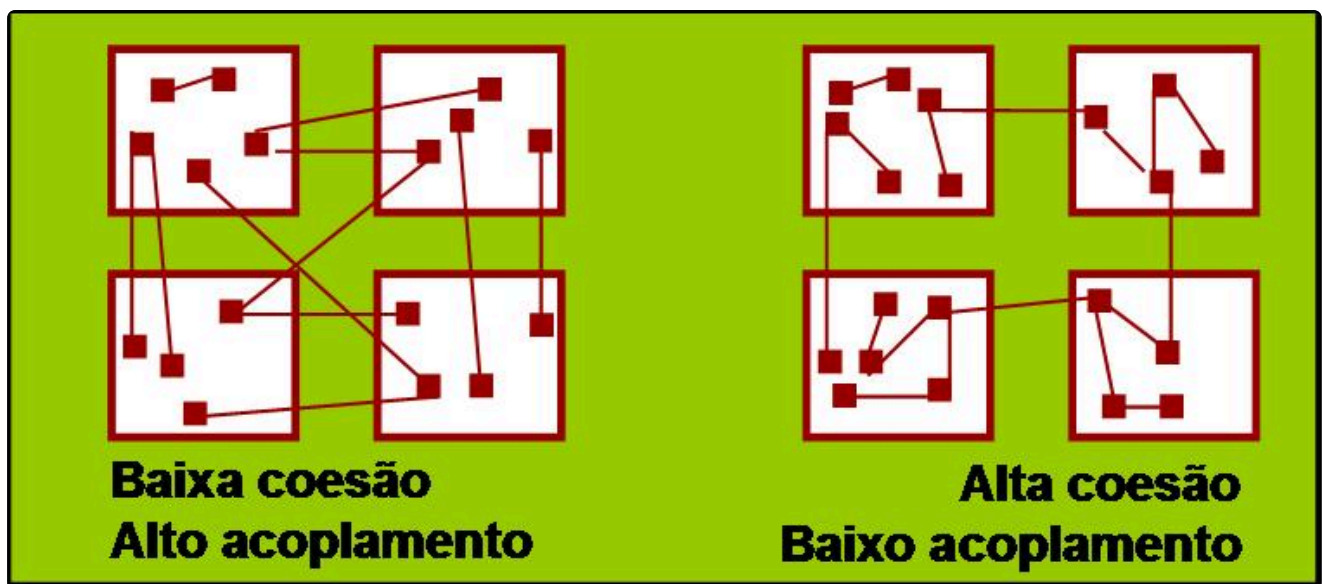
- Herança é uma forma de reuso caixa branca (a subclasse enxerga os detalhes internos da superclasse).
- Composição é uma forma de reuso caixa preta (**quem usa o objeto só conhece a interface**).
- Herança e composição são usadas em conjunto num mesmo sistema, não são mutuamente exclusivas.
- Deve-se favorecer composição em relação à herança.

Só que a **composição também tem um custo**:

- A composição aumenta a complexidade estrutural do sistema (mais classes, mais objetos colaborando).
- **A composição aumenta o acoplamento entre essas classes, ainda que de um tipo mais controlado do que o da herança.**

Isso leva às duas perguntas que abrem a aula: por que acoplamento é tão problemático, e qual o tipo de acoplamento que realmente se quer ou precisa.

2. Acoplamento: o problema central



2.1 Um código concreto para analisar

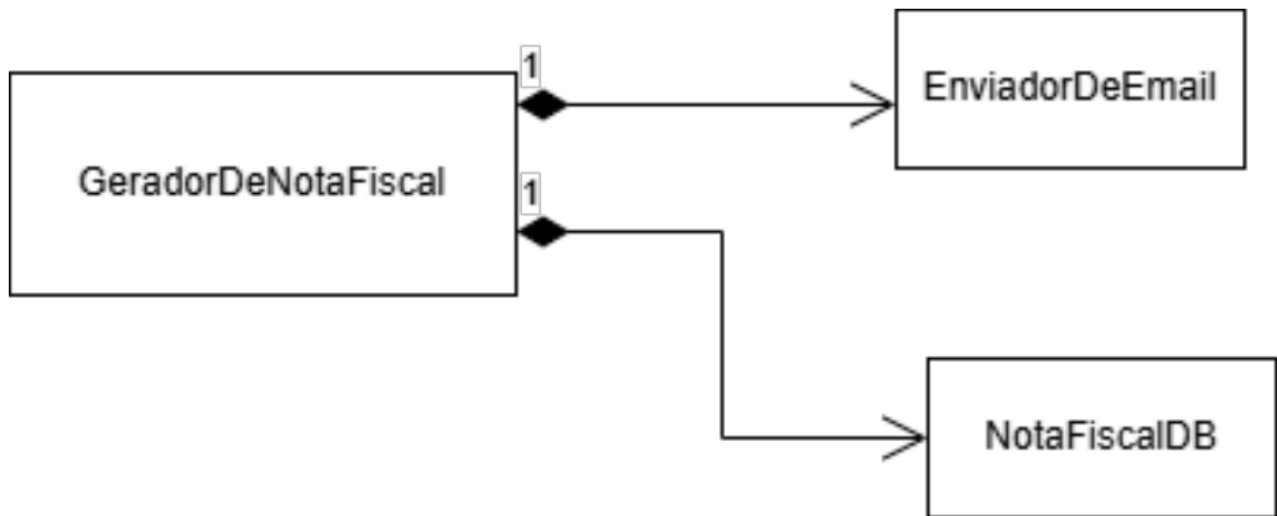
```
public class GeradorDeNotaFiscal {  
    private final EnviadorDeEmail email;  
    private final NotaFiscalDB db;  
  
    public GeradorDeNotaFiscal(EnviadorDeEmail email, NotaFiscalDB db) {  
        this.email = email;  
        this.db = db;  
    }  
  
    public NotaFiscal gera(Fatura fatura) {  
        double valor = fatura.getValorMensal();  
        NotaFiscal nf = new NotaFiscal(valor,  
            impostoSimplesSobre0(valor));  
        email.enviaEmail(nf);  
        db.persiste(nf);  
        return nf;  
    }  
  
    private double impostoSimplesSobre0(double valor) {  
        return valor * 0.06;  
    }  
}
```

`GeradorDeNotaFiscal` já usa composição: depende de `EnviadorDeEmail` e de `NotaFiscalDB` como colaboradores, em vez de herdar deles ou fazer tudo sozinho. Mesmo assim, o problema aparece:

o grande problema é que qualquer mudança em qualquer uma dessas duas classes pode impactar em mudanças na classe principal. Se a assinatura de `enviaEmail` mudar, ou se `NotaFiscalDB` for substituído por outra forma de persistência,

`GeradorDeNotaFiscal` sente o impacto diretamente, porque depende das classes concretas.

O diagrama correspondente:



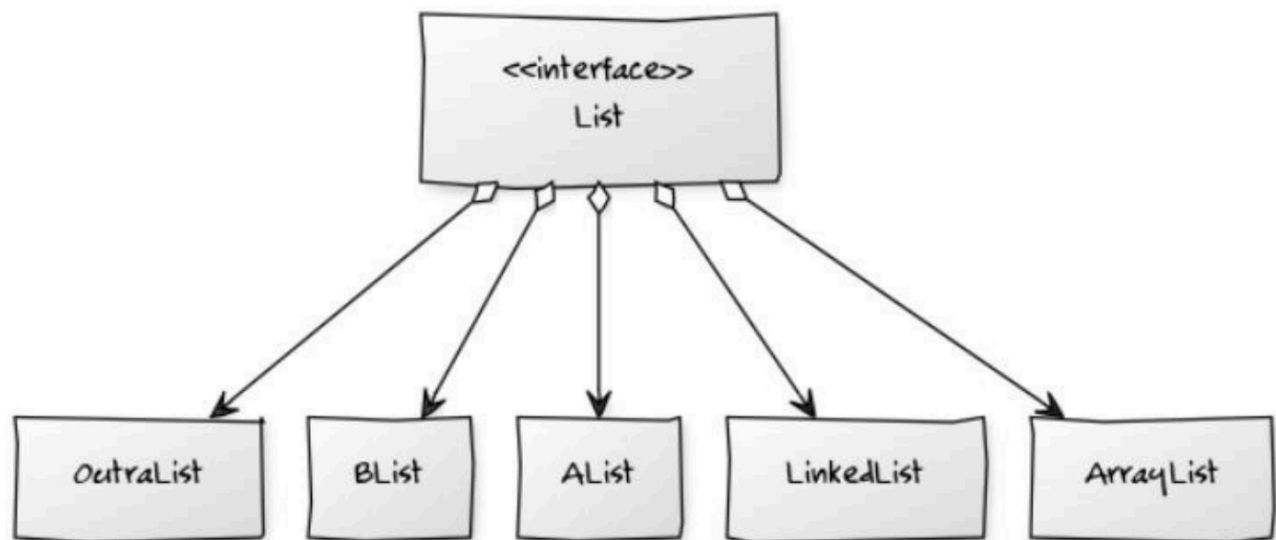
`GeradorDeNotaFiscal` está ligado por composição (losango preenchido) diretamente às duas classes concretas, com multiplicidade 1 para cada uma.

2.2 Nem todo acoplamento é igualmente problemático

Diante disso, cabe perguntar: será que dá para acabar com o acoplamento? Dá para fazer com que as classes não dependam de nada?

Não é bem essa a saída, porque todo código depende de alguma coisa. O próprio `GeradorDeNotaFiscal` já usa `String`, e um sistema típico usa a interface `List` o tempo todo, e ninguém considera esses acoplamentos um problema sério. Por outro lado, o acoplamento com `EnviadorDeEmail` (uma classe concreta e específica do domínio) já causa desconforto. Ou seja: **nem todo acoplamento é problemático da mesma forma**. A questão vira: quais características de `List` e de `String` fazem esse tipo de acoplamento ser menos arriscado?

A resposta que a aula constrói: acoplar com módulos estáveis, que tendem a mudar pouco, dói muito menos do que acoplar com módulos voláteis. `List` é uma interface da biblioteca padrão, com contrato firmado há décadas; qualquer classe concreta que a implemente (`ArrayList`, `LinkedList`, e outras) pode ser trocada livremente por quem usa `List` como tipo.

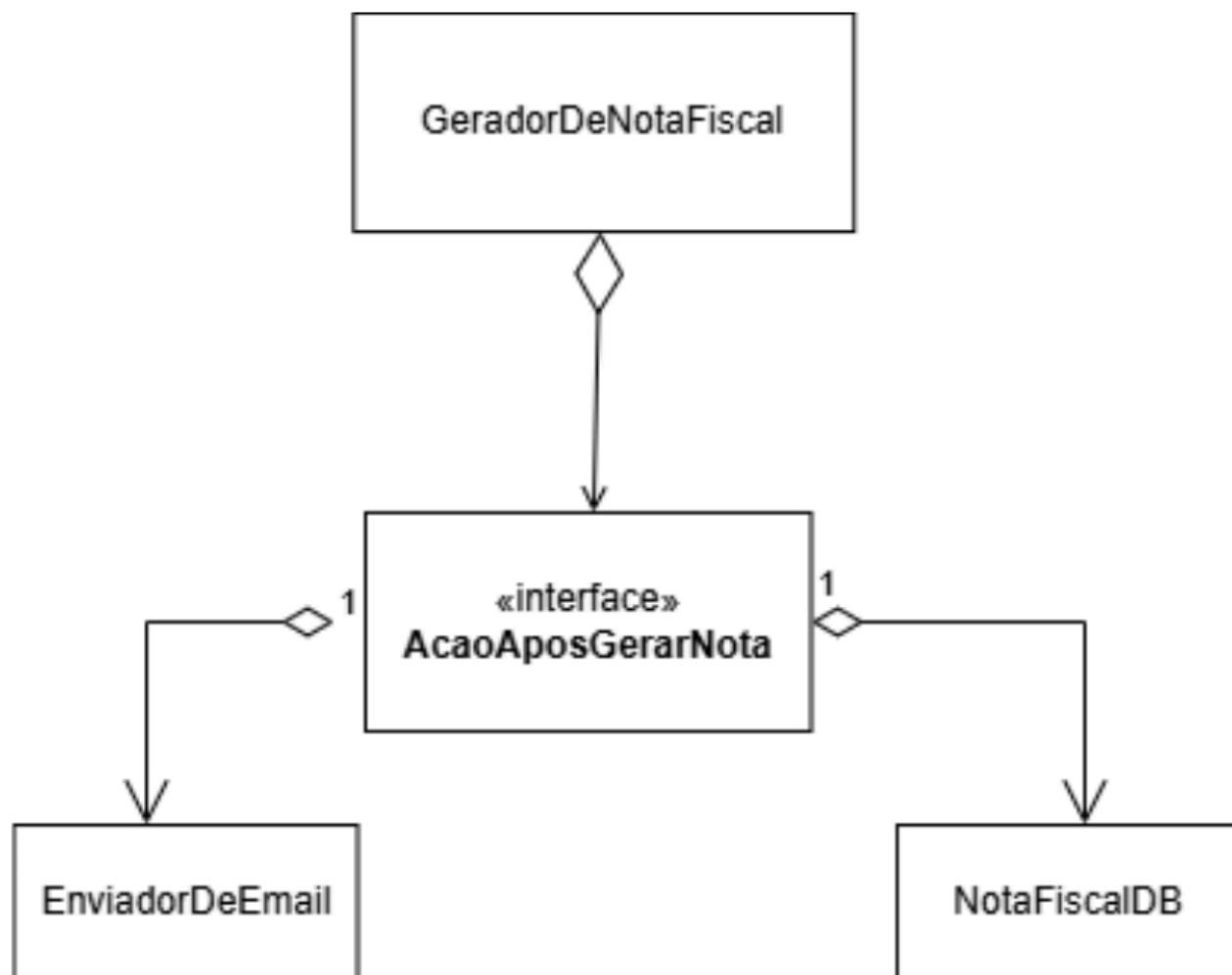


2.3 A ideia de reduzir o acoplamento acoplando mais, só que a módulos estáveis

A proposta é: se for criada uma interface `AcaoAposGerarNota`, e cada ação que deve ocorrer depois de gerar a nota fiscal implementar essa interface, então

`GeradorDeNotaFiscal` passa a depender só dessa interface, e não mais das classes concretas de e-mail e de banco de dados.

A interface é estável por natureza, porque não muda toda vez que aparece uma nova ação a executar; quem muda é a lista de implementações por trás dela (até porque uma classe que implementa uma interface **pode** ter novos métodos que não estão nela (embora eles não possam ser chamados pelo nome da interface)).



A frase que resume a estratégia: a ideia para reduzir o problema do acoplamento é começar a acoplar mais com módulos estáveis, que tendem a mudar menos.

2.4 O código refatorado

```
public class GeradorDeNotaFiscal {
    private final List<AcaoAposGerarNota> acoes;

    public GeradorDeNotaFiscal(List<AcaoAposGerarNota> acoes) {
        this.acoes = acoes;
    }

    public NotaFiscal gera(Fatura fatura) {
        double valor = fatura.getValorMensal();
        NotaFiscal nf = new NotaFiscal(valor,
        impostoSimplesSobre0(valor));
        for (AcaoAposGerarNota acao : acoes) {
            acao.executa(nf);
        }
        return nf;
    }

    private double impostoSimplesSobre0(double valor) {
        return valor * 0.06;
    }
}
```

```

    }
}

interface AcaoAposGerarNota {
    void executa(NotaFiscal nf);
}

class NotaFiscalDB implements AcaoAposGerarNota {
    // implementação
}

class EnviadorDeEmail implements AcaoAposGerarNota {
    // implementação
}

```

Note que `GeradorDeNotaFiscal` não conhece mais `EnviadorDeEmail` nem `NotaFiscalDB` por nome: conhece só a lista de `AcaoAposGerarNota`. Adicionar uma terceira ação (gerar um log, por exemplo) não exige tocar em `GeradorDeNotaFiscal`, só criar mais uma classe que implemente a interface e incluí-la na lista.

3. O padrão Observer

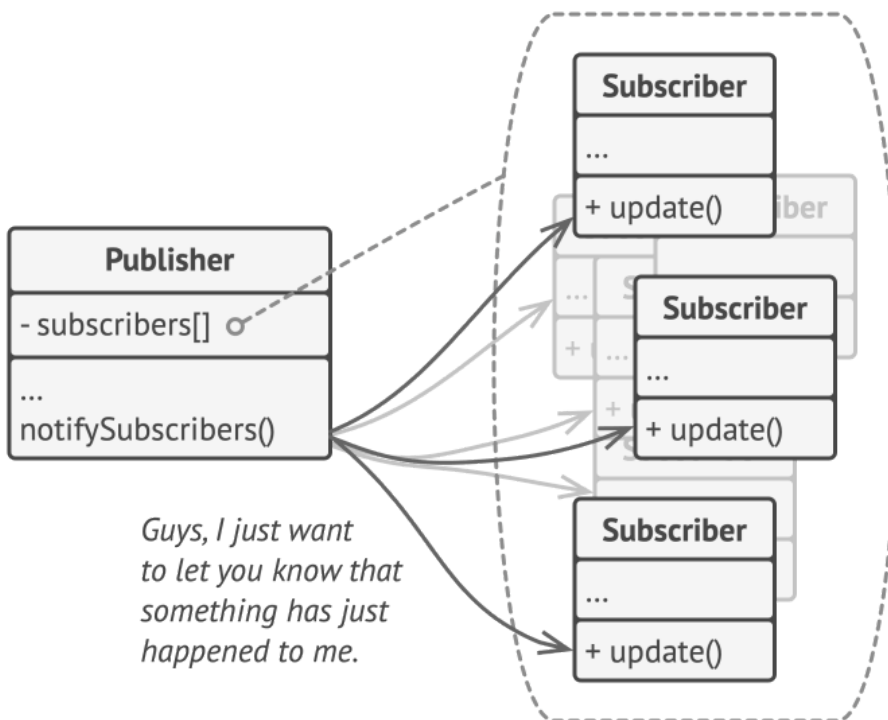
Esse é o padrão comportamental que dá nome estruturado a essa ideia de acoplar objetos que reagem a mudanças de estado sem que o objeto observado precise conhecer quem são eles.

3.1 Definição

O padrão Observer permite que **objetos interessados sejam avisados da mudança de estado ou de eventos que ocorrem em outro objeto**. Essa relação também é chamada de *publish-subscribe*: quem sofre a mudança publica um evento, e quem está inscrito recebe a notificação.



O padrão define uma dependência um-para-muitos entre objetos, de forma que quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente (GAMMA et al., 2000). Dois papéis ficam definidos:

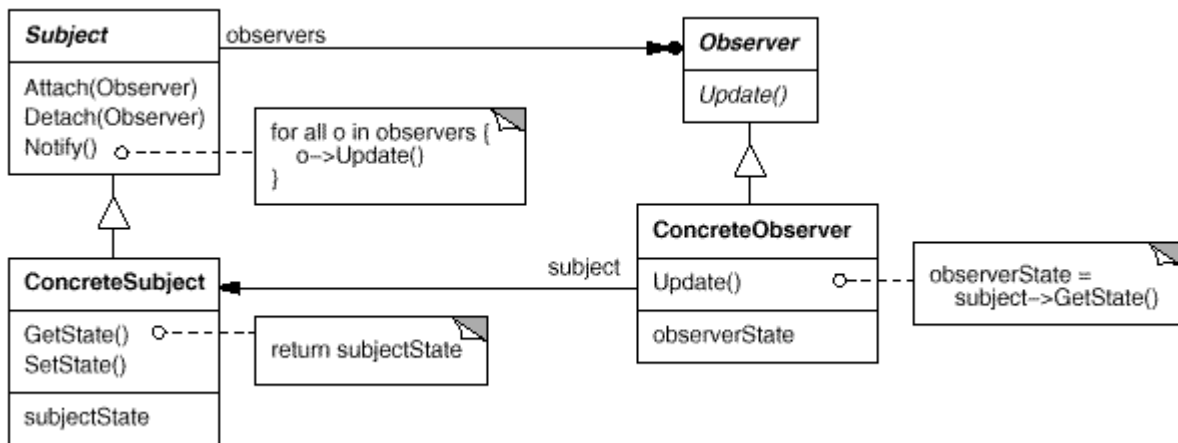


- **Subject** (o objeto observado): lança eventos, é quem detém a informação e quem conhece a lista de interessados.
- **Observer** (o objeto que observa): recebe e processa os eventos, sem que o Subject precise saber os detalhes de cada observador concreto.

O Subject é modelado como uma abstração estável, tipicamente uma interface, exatamente pela mesma razão do `AcaoAposGerarNota`: o contrato de "quem pode se inscrever e como é avisado" muda muito pouco, ainda que as classes concretas que observam mudem com frequência. O Subject conhece só a classe base `Observer`, nunca as classes concretas que a implementam; é isso que garante o baixo acoplamento entre os dois lados (frequentemente chamado de acoplamento leve, ou *loose coupling*).

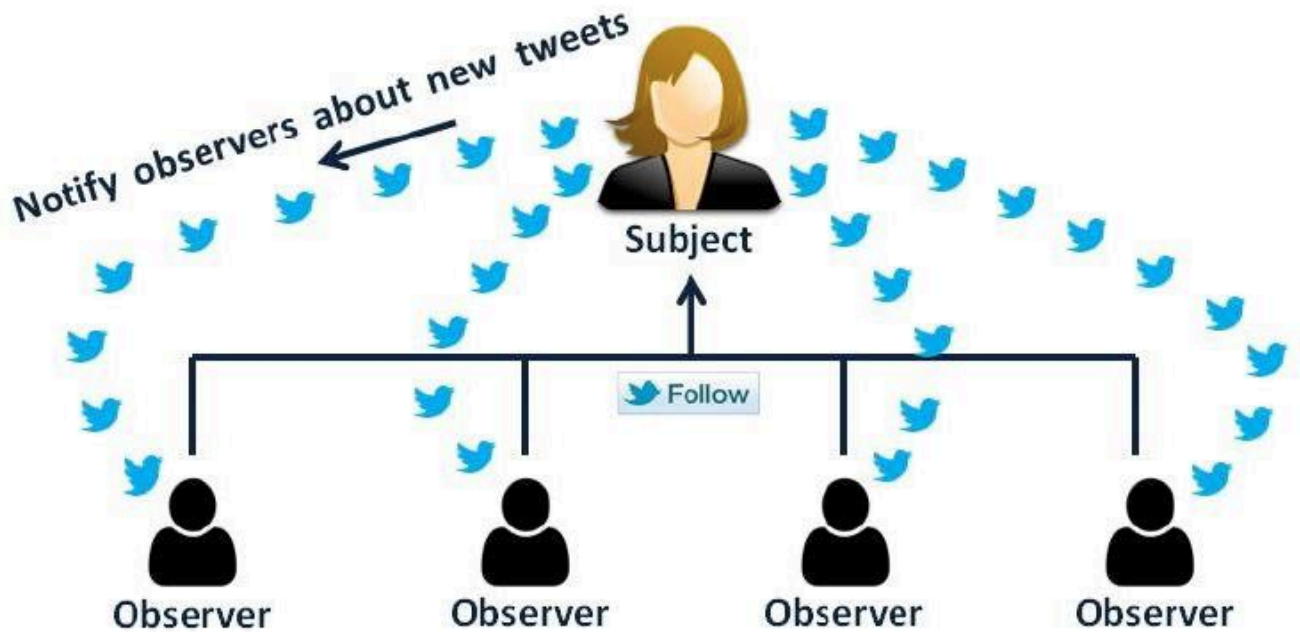
Esse é justamente o "poder da ligação leve" descrito no livro: quando dois objetos estão levemente ligados, eles podem interagir, mas sabem muito pouco um sobre o outro. O Subject não precisa conhecer a classe concreta do observador, nem o que ela faz internamente; e como a única coisa da qual o Subject depende é uma lista de objetos que respeitam a interface `Observer`, é possível adicionar, remover ou substituir observadores livremente, inclusive em tempo de execução, sem jamais alterar o código do Subject (FREEMAN; FREEMAN, 2009). Essa é a razão de o livro elevar isso a um princípio de projeto geral: buscar sempre o menor acoplamento possível entre os objetos que interagem, porque projetos levemente ligados são mais flexíveis e mais fáceis de estender (FREEMAN; FREEMAN, 2009).

3.2 Estrutura genérica (UML)



Participante	Papel
Subject	Conhece seus observadores (qualquer número deles). Fornece a interface para registrar e remover observadores (<code>Attach / Detach</code> no diagrama do livro; <code>registerObserver / removeObserver</code> no vocabulário da aula).
Observer	Define a interface de atualização (<code>Update</code>) para os objetos que devem ser notificados quando o <code>Subject</code> muda.
ConcreteSubject	Guarda o estado de interesse dos observadores concretos e dispara a notificação (<code>Notify</code>) sempre que esse estado muda.
ConcreteObserver	Mantém uma referência ao <code>ConcreteSubject</code> e um estado próprio, que deve ficar consistente com o do <code>Subject</code> ; implementa <code>Update</code> para isso.

Observer Design Pattern



A colaboração central: o ConcreteSubject notifica seus observadores sempre que uma mudança pode tornar o estado deles inconsistente com o seu; depois de ser avisado, o ConcreteObserver pode consultar o Subject para obter as informações de que precisa e realinhar seu próprio estado (GAMMA et al., 2000).

3.3 Como fazer, em sete passos

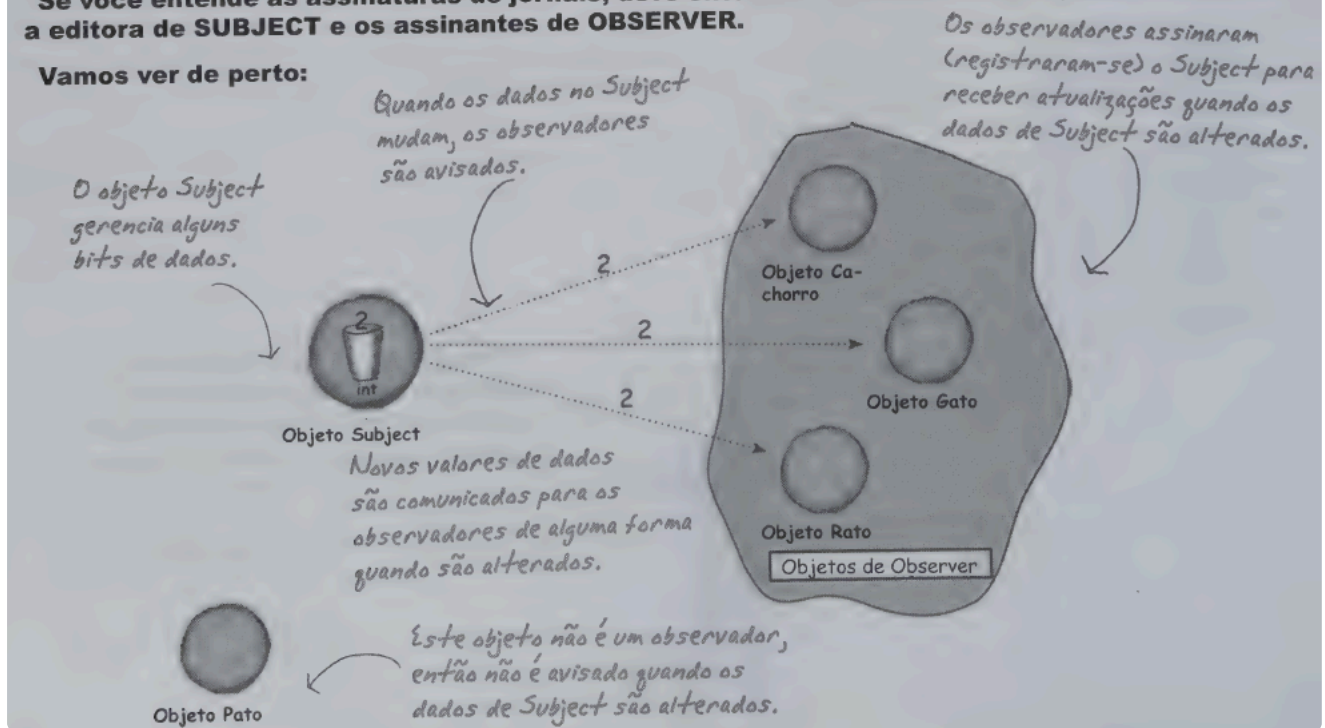
A aula descreve o mesmo esqueleto do quadro anterior como uma sequência de decisões de projeto:

1. Modelar a funcionalidade "independente" (o dado que muda) com uma abstração de Subject.
2. Modelar a funcionalidade "dependente" (quem reage à mudança) com uma hierarquia de Observer.
3. O Subject fica acoplado apenas à classe base Observer, nunca às classes concretas.
4. Os observadores se registram no Subject (é o próprio observador que pede para entrar na lista).
5. O Subject transmite eventos para todos os observadores registrados.
6. Os observadores "buscam" no Subject as informações de que precisam (mais sobre essa escolha na seção 8).
7. Quem configura o número e o tipo de observadores é o cliente do sistema, isto é, o código que monta o cenário (o `main`, ou uma camada de inicialização), e não o Subject nem o Observer.

Editora + Assinantes = Padrão Observer

Se você entende as assinaturas de jornais, deve entender o Padrão Observer, onde chamamos a editora de **SUBJECT** e os assinantes de **OBSERVER**.

Vamos ver de perto:

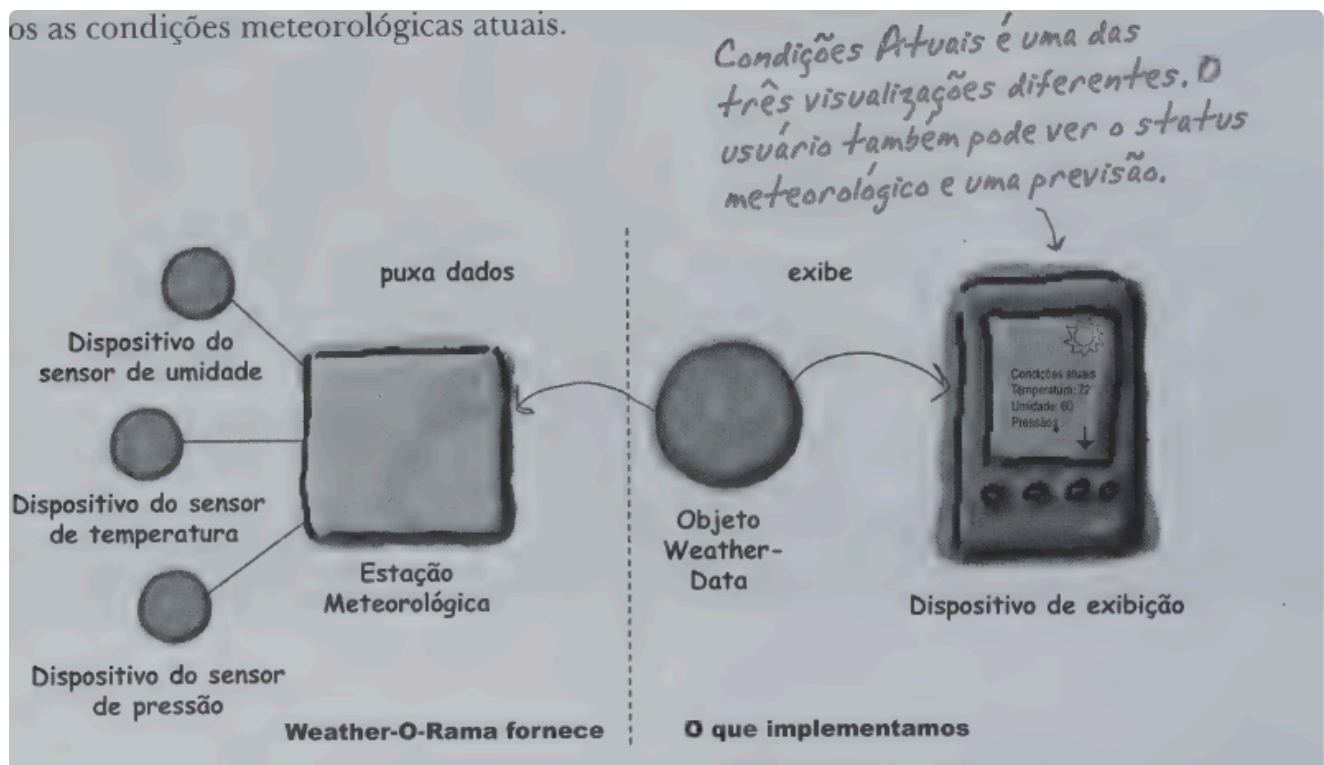


4. Exemplo: estação meteorológica

4.1 O cenário

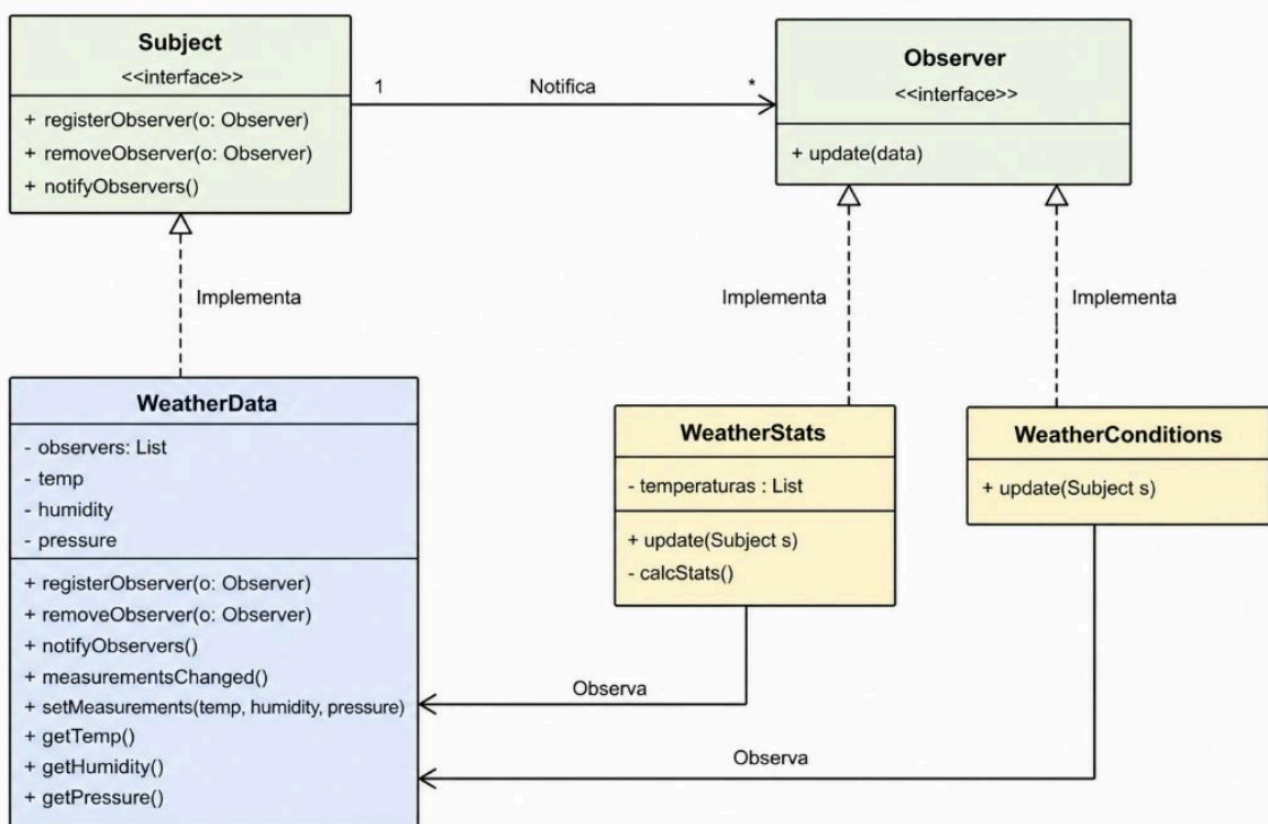
Existe uma estação meteorológica responsável por fornecer dados de temperatura, umidade e pressão. Duas outras entidades precisam desses dados sempre atualizados: a cada nova leitura da estação, ambas precisam ser informadas. Na aula, essas entidades ficam responsáveis por apresentar na tela dos usuários os dados atualizados e os resultados de cálculos feitos em cima deles (uma média, por exemplo).

os as condições meteorológicas atuais.



Esse é o mesmo exemplo clássico do livro Use a Cabeça! Padrões de Projetos, onde a WeatherData (o equipamento que mede as condições do tempo) faz o papel de Subject, e cada display (painel de condições atuais, painel de estatísticas, previsão) faz o papel de Observer (FREEMAN; FREEMAN, 2009).

4.2 Diagrama de classes



Lendo o diagrama: Subject e Observer são interfaces. WeatherData implementa Subject e concentra o estado (temperatura, umidade, pressão) e a lista de observadores.

WeatherStats e WeatherConditions implementam Observer , cada uma reagindo à mesma notificação de um jeito diferente (uma calcula estatísticas, a outra só exibe as condições correntes).

4.3 As interfaces

```
package map_observer;

public interface Subject {
    public void registerObserver(Observer o);
    public void removeObserver(Observer o);
    public void notifyObservers();
}
```

```
package map_observer;

public interface Observer {
    void update(Subject s);
}
```

4.4 O Subject concreto: WeatherData

```
package map_observer;

import java.util.ArrayList;

public class WeatherData implements Subject {

    private double temp;
    private double humidity;
    private double pressure;
    private List<Observer> observers;

    // Não registro os observers aqui
    public WeatherData() {
        this.observers = new ArrayList<Observer>();
    }

    @Override
    public void registerObserver(Observer o) {
        observers.add(o);
    }

    @Override
    public void notifyObservers() {
        for (Observer o : observers) {
```



```
s).getPressure());
    }
}
```

O comentário ao lado desse trecho no slide resume bem o passo 6 da lista da seção 3.3: **os observadores "buscam" as informações de que precisam do Subject**, fazendo o downcast de `Subject` para `WeatherData` e chamando os getters (`getTemp`, `getHumidity`, `getPressure`) conforme a necessidade de cada um.

```
package map_observer;

import java.util.ArrayList;
import java.util.List;

public class WeatherStats implements Observer {

    private List<Double> temperaturas = new ArrayList<>();

    @Override
    public void update(Subject s) {
        temperaturas.add(((WeatherData) s).getTemp());
        calcStats(); // sempre recalculo
    }

    private void calcStats() {
        double soma = 0;
        for (double t : temperaturas) {
            soma += t;
        }
        double media = soma / temperaturas.size();

        double min = temperaturas.get(0);
        double max = temperaturas.get(0);
        for (double t : temperaturas) {
            if (t < min) min = t;
            if (t > max) max = t;
        }
        // média, mínimo e máximo calculados a cada atualização
    }
}
```

`WeatherStats` guarda seu próprio histórico de temperaturas (o estado dela é diferente do estado do `Subject`) e recalcula estatísticas simples a cada notificação. Isso ilustra bem por que cada observador tem seu próprio `update` -> `WeatherConditions` só exibe o dado corrente, enquanto `WeatherStats` acumula e processa.

4.6 Registrando os observadores e simulando leituras

```

package map_observer;

public class MainClass {
    public static void main(String[] args) throws Exception {
        WeatherConditions wcond = new WeatherConditions(); // Observer
        WeatherStats ws = new WeatherStats();                // Observer
        WeatherData wd = new WeatherData();                  // Subject

        wd.registerObserver(wcond);
        wd.registerObserver(ws);
    }
}

```

E, mais à frente, o cliente também é responsável por alimentar o Subject com novas leituras ao longo do tempo:

```

package map_observer;

public class MainClass {
    public static void main(String[] args) throws Exception {
        WeatherConditions wcond = new WeatherConditions(); // Observer
        WeatherStats ws = new WeatherStats();                // Observer
        WeatherData wd = new WeatherData();                  // Subject

        wd.registerObserver(wcond);
        wd.registerObserver(ws);
        wd.setMeasurements(25, 60, 1011);
        Thread.sleep(2000);
        wd.setMeasurements(26, 62, 1010);
        Thread.sleep(2000);
        wd.setMeasurements(27, 60, 1010);
        Thread.sleep(2000);
        wd.setMeasurements(15, 30, 1013);
        Thread.sleep(2000);
        wd.setMeasurements(40, 45, 1005);
    }
}

```

Cada chamada a `setMeasurements` dispara `measurementsChanged`, que chama `notifyObservers`, que por sua vez chama `update` em `wcond` e em `ws`; os dois reagem à mesma notificação de formas diferentes, sem que `WeatherData` precise saber disso. É exatamente o "cliente configura o número e o tipo de observadores" do passo 7: quem decide que existem dois observadores, e quais são eles, é o `main`, não a estação.

5. Herança e composição de novo: por que o Observer é composição

O Observer só funciona porque `WeatherData` tem uma lista de `Observer` (agregação, relação "tem um", losango vazado por se tratar de uma coleção de referências que existem independentemente do Subject) e não porque `WeatherConditions` e `WeatherStats` herdam de `WeatherData`. Se a solução fosse por herança, cada novo tipo de display exigiria uma nova subclasse de `WeatherData`, reproduzindo o mesmo problema de explosão de subclasses visto no exemplo do frete que o strategy resolve. Com agregação sobre a interface `Observer`, um novo display é só mais uma classe que implementa `update`, registrada em tempo de execução, sem tocar em `WeatherData`.

6. Empurrar ou puxar dados: push e pull

Uma decisão de projeto é como o observador recebe o estado do Subject: por empurrar (push) ou por puxar (pull).

- **Modelo push:** o Subject manda, junto da notificação, todos os dados que julga relevantes, quer os observadores precisem de tudo isso ou não. Vantagem: o observador recebe tudo em uma única chamada. Desvantagem: o Subject passa a supor o que os observadores precisam, o que pode deixar a interface de atualização inchada e menos reutilizável (FREEMAN; FREEMAN, 2009).
- **Modelo pull:** o Subject manda a notificação mais enxuta possível (às vezes só avisando "eu, o Subject, mudei"), e cada observador busca, por conta própria, exatamente os dados de que precisa, usando getters do Subject. Vantagem: o Subject não precisa antecipar as necessidades de cada observador, e é mais fácil evoluir o Subject sem quebrar a interface de atualização; se surgirem novos dados no Subject, basta acrescentar novos getters, sem alterar a assinatura de `update` nem o código dos observadores existentes. Desvantagem: pode ser um pouco menos eficiente, porque exige chamadas extra de método (FREEMAN; FREEMAN, 2009).

O exemplo da estação segue o modelo pull: `update(Subject s)` recebe só a referência ao Subject, e cada observador chama `getTemp()`, `getHumidity()` ou `getPressure()` conforme sua necessidade. O pull é geralmente considerado a opção mais "correta" justamente por preservar melhor o desacoplamento entre Subject e Observer.

Um detalhe importante é que: mesmo que um observador não use todos os dados que mudaram, ele é notificado de qualquer forma, porque a notificação é um broadcast (a mesma chamada de `notifyObservers` chega a todos os registrados). O que muda entre push e pull é só a quantidade de informação que vem embutida nessa notificação; a notificação em si sempre alcança todos os observadores registrados, sejam eles interessados ou não naquele dado específico. O próprio GoF chama atenção para o preço

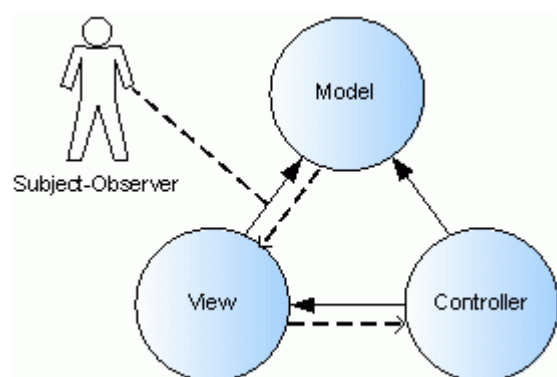
disso: como um observador não sabe da existência dos outros, uma operação aparentemente inofensiva no Subject pode disparar uma cascata de atualizações em cadeia, e por isso é sempre boa prática documentar quais operações de um Subject disparam notificações (GAMMA et al., 2000).

7. O padrão Observer embutido em Java e em outros frameworks

O pacote `java.util` chegou a oferecer uma implementação genérica do padrão, com a interface `Observer` e a classe `Observable`, bastante parecidas com o par `Subject / Observer` construído à mão acima, mas com mais recursos prontos, incluindo suporte tanto a push quanto a pull (FREEMAN; FREEMAN, 2009). Essa dupla foi descontinuada a partir do Java 9 (está marcada como deprecated desde então), justamente por limitações de projeto, como não permitir que `Observable` fosse uma interface (é uma classe, o que consome a única herança disponível de quem quisesse ser ao mesmo tempo `Subject` e outra coisa); construir a própria interface, continua sendo a recomendação atual.

O padrão também está por trás de mecanismos bem conhecidos: os *listeners* de eventos do Swing (um `ActionListener` registrado em um botão é um `Observer`; o botão é o `Subject`) e, de forma mais ampla, de qualquer framework de interface gráfica ou de eventos que siga o modelo *publish-subscribe*.

8. Observer dentro do MVC



No MVC, o Model faz o papel de Subject e a View faz o papel de Observer: o Model não conhece a View, a View se registra para observar o Model, e mudanças no Model são notificadas automaticamente para todas as Views registradas, exatamente como `WeatherData` notifica `WeatherConditions` e `WeatherStats`. É por isso que o mesmo Model pode alimentar várias interfaces diferentes ao mesmo tempo (um app, um site, um painel administrativo), sem que o Model precise saber quantas nem quais existem.

Vale registrar que o primeiro uso amplamente conhecido do padrão Observer foi justamente no MVC originado no Smalltalk, onde a classe Model cumpre o papel de Subject e View é a classe base dos observadores (GAMMA et al., 2000).

9. Vantagens e consequências

O padrão permite variar Subject e observadores de forma independente: dá para reutilizar um sem reutilizar o outro, e dá para acrescentar novos observadores sem alterar o Subject nem os observadores já existentes (GAMMA et al., 2000). Isso traz alguns efeitos que vale ter em mente:

- **Acoplamento abstrato e mínimo:** tudo que o Subject sabe é que existe uma lista de objetos que seguem a interface Observer; ele nunca conhece a classe concreta de nenhum deles.
 - **Suporte nativo a broadcast:** a notificação não precisa especificar um destinatário; ela chega automaticamente a todos os interessados que se inscreveram, e cabe a cada observador decidir se vai tratar ou ignorar aquele evento.
 - **Atualizações inesperadas:** como cada observador ignora a existência dos demais, uma mudança aparentemente pequena no Subject pode gerar uma cascata de atualizações em vários observadores ao mesmo tempo; sem um protocolo claro sobre o que mudou, os observadores podem ter que trabalhar mais do que o necessário só para descobrir o que de fato importa para eles (GAMMA et al., 2000).
-

10. Perguntas de fechamento

Observer serve só para atualizar status? Na prática, atualizar o estado exibido em tela é o uso mais comum e mais intuitivo (foi o próprio exemplo da estação), mas o padrão é mais geral do que isso: ele serve para propagar qualquer mudança de estado ou qualquer evento de interesse de um objeto para um conjunto de dependentes, o que inclui disparar cálculos (caso de WeatherStats), registrar logs, enviar notificações externas, ou qualquer reação a um evento que não seja necessariamente "atualizar uma tela".

Isso fecha o ciclo com o problema do início da aula: tanto a solução do GeradorDeNotaFiscal quanto o Observer resolvem o mesmo problema de fundo (acoplamento excessivo a classes concretas voláteis) com a mesma ferramenta de fundo (composição sobre uma interface estável); o Observer acrescenta a esse esqueleto o vocabulário e a estrutura específicos para o caso em que existem muitos dependentes reagindo à mesma mudança de estado.

11. Exercício proposto: Observer no iFood

Implementar o padrão Observer em Java simulando o funcionamento de um sistema de pedidos parecido com o iFood, em que diferentes partes do sistema são notificadas automaticamente quando o status de um pedido é atualizado.

Um esqueleto inicial, seguindo a mesma estrutura da estação meteorológica, como ponto de partida para o exercício:

```
interface Subject {
    void registerObserver(Observer o);
    void removeObserver(Observer o);
    void notifyObservers();
}

interface Observer {
    void update(Subject s);
}

class Pedido implements Subject {
    private String status; // ex.: "Confirmado", "Em preparo", "A
caminho", "Entregue"
    private List<Observer> observers = new ArrayList<>();

    public void registerObserver(Observer o) { observers.add(o); }
    public void removeObserver(Observer o) { observers.remove(o); }
    public void notifyObservers() {
        for (Observer o : observers) o.update(this);
    }

    public void atualizaStatus(String novoStatus) {
        this.status = novoStatus;
        notifyObservers();
    }

    public String getStatus() { return status; }
}
```